Phase 5 Notes: Generating a BST from Preorder Traversal and Understanding the Main Drawback

1. Where You Are Now

Before Phase 5, you already learned the main operations of a Binary Search Tree, also called a BST.

Phase 1: Binary Tree Foundations

You learned that a binary tree is made of nodes.

Each node has three parts:

```
struct Node {
    struct Node *lchild;
    int data;
    struct Node *rchild;
};
```

Meaning:

- `lchild` stores the address of the left child.
- `data` stores the value inside the node.
- `rchild` stores the address of the right child.
- `NULL` means there is no node in that direction.

You also learned:

- The root is the top node.
- A child is a node connected below another node.
- A subtree is a smaller tree inside a bigger tree.
- A leaf is a node with no children.
- Recursion works well with trees.
- `height()` finds the longest path downward.
- Tree algorithms often depend on height.

Phase 2: BST Searching

You learned the BST rule:

```
left subtree values < node value < right subtree values
```

When searching in a BST:

```
key == current node data  -> found
key < current node data   -> go left
key > current node data   -> go right
current pointer == NULL   -> not found
```

The main idea was:

```
A BST search follows only one path, not the whole tree.
```

Phase 3: Iterative Insertion

You learned that insertion is like searching first.

To insert a value:

1. Start at the root.
2. Compare the new key with the current node.
3. Move left or right using the BST rule.
4. If the key already exists, stop.
5. If you reach `NULL`, insert the new node there.

You also learned about two pointers:

```
t = current pointer
r = parent pointer
```

The pointer `t` moves down the tree. The pointer `r` remembers the parent.

This is needed because when `t` becomes `NULL`, we need the parent node to attach the new node.

Phase 4: Recursive Insertion and Recursive Deletion

You learned recursive insertion and deletion.

The most important idea was:

```
Every recursive insert or delete call returns the updated root of that subtree.
```

For recursive insertion:

```
p->lchild = RInsert(p->lchild, key);
p->rchild = RInsert(p->rchild, key);
```

For recursive deletion:

```
p->lchild = Delete(p->lchild, key);
p->rchild = Delete(p->rchild, key);
```

You also learned deletion cases:

- Leaf node: delete it and return `NULL`.
- One child: delete the node and return its child.
- Two children: replace using inorder predecessor or inorder successor.

Your Current Level

At this point, you should be able to:

- Create a BST.
- Search in a BST.
- Insert into a BST.
- Traverse a BST using in-order traversal.
- Delete from a BST.
- Understand why height affects BST speed.

Now Phase 5 teaches two final ideas:

1. How to generate a BST from preorder traversal.
2. Why ordinary BSTs can become slow.

---

2. Main Goal of Phase 5

Phase 5 has two big goals.

Goal 1: Generate a BST from Preorder Traversal

You will learn how to build a BST when you are given only its preorder traversal.

Example preorder:

```
30, 20, 10, 15, 25, 40, 50, 45
```

From this list, we will build the original BST.

Goal 2: Understand the Main Drawback of Ordinary BSTs

You will learn that a normal BST does not control its own height.

This means:

```
The same values can create a fast BST or a slow BST depending on insertion
order.
```

That is why self-balancing trees like AVL trees exist.

---

3. Important Review: BST Rule

Before building a BST from preorder, remember the BST rule.

For every node:

```
All smaller values go to the left.
All larger values go to the right.
```

Another way:

```
left subtree values < node value < right subtree values
```

Example:

```
    30
   /
 20    40
```

This is a valid BST because:

```
20 < 30
40 > 30
```

A more detailed example:

```
    30
   /
 20     40

   25
```

This is also valid because:

```
25 > 20
25 < 30
```

So `25` belongs in the right subtree of `20`, but it is still inside the left subtree of `30`.

This idea is very important in Phase 5 because when we build from preorder, we must respect ancestor limits.

---

4. What Is Preorder Traversal?

Preorder traversal visits a tree in this order:

```
Node, Left, Right
```

That means:

1. Visit the current node first.
2. Visit the left subtree.
3. Visit the right subtree.

Short form:

```
N L R
```

Where:

```
N = Node
L = Left subtree
R = Right subtree
```

Example tree:

```
        30
       /
     20     40
    /
  10   25
```

Preorder traversal:

```
30, 20, 10, 25, 40
```

Why?

Start at root:

```
30
```

Then go left subtree:

```
20, 10, 25
```

Then go right subtree:

```
40
```

So full preorder is:

```
30, 20, 10, 25, 40
```

---

5. Why the First Preorder Value Becomes the Root

In preorder traversal, the current node is visited before its left and right subtrees.

So the first value in preorder must be the root.

Example:

```
Preorder: 30, 20, 10, 15, 25, 40, 50, 45
```

The first value is:

```
30
```

So the root is:

```
30
```

Initial tree:

```
30
```

Then the remaining values are placed using the BST rule.

---

6. Can Preorder Alone Build Any Binary Tree?

No.

Preorder alone is not enough to rebuild every normal binary tree.

Why?

Because a normal binary tree has no sorting rule.

Example preorder:

```
10, 20, 30
```

This preorder could represent different binary trees.

Tree 1:

```
10
/
20
```

```
/
30
```

Tree 2:

```
10

 20

  30
```

Tree 3:

```
  10
 /
20

 30
```

So preorder alone does not uniquely identify a general binary tree.

But for a BST, we have an extra rule:

```
smaller values go left
larger values go right
```

That rule gives enough guidance to build the BST.

So remember:

```
Preorder alone cannot build every binary tree.
Preorder can build a BST because the BST rule gives extra information.
```

---

7. Why a BST Can Be Built from Preorder Alone

A BST has two useful properties.

Property 1: The first preorder value is the root

Because preorder is:

```
Node, Left, Right
```

The first value is the root.

Property 2: BST values follow a sorted rule

For every node:

```
left values < node value < right values
```

Also, the in-order traversal of a BST is always sorted.

In-order traversal is:

```
Left, Node, Right
```

For a BST, this prints values in ascending order.

Example:

```
        30
       /
     20    40
    /
  10   25
```

In-order output:

```
10, 20, 25, 30, 40
```

This sorted property helps us reason about where preorder values belong.

---

8. Simple Way vs Stack-Based Way

There are two ways to build a BST from preorder values.

Method 1: Insert one by one

You can take each preorder value and insert it normally into the BST.

Example:

```
30, 20, 10, 15, 25, 40, 50, 45
```

You could do:

```
insert(30)
insert(20)
insert(10)
insert(15)
insert(25)
insert(40)
insert(50)
insert(45)
```

This works.

But it may take more time if the tree becomes skewed.

Method 2: Use a stack

Phase 5 teaches the stack-based method.

This method reads the preorder array once from left to right.

It uses a stack to remember ancestors.

The stack helps when we go deep left and later need to come back upward to attach right children.

---

9. Why We Need a Stack

Imagine this preorder:

```
30, 20, 10, 15, 25
```

We start:

```
30
```

Then `20` is smaller than `30`, so it goes left.

```
  30
 /
20
```

Then `10` is smaller than `20`, so it goes left.

```
    30
   /
 20
 /
10
```

Now `15` is greater than `10`, so it can be right child of `10`.

```
    30
   /
 20
 /
10

  15
```

Now the next value is `25`.

`25` is greater than `15`, but it cannot be right child of `15` because `15` is inside the left subtree of `20`.

Everything inside the left subtree of `20` must be less than `20`.

But:

```
25 > 20
```

So we need to go back up to node `20`.

The stack helps us remember `20` and `30` while we are deep in the left side.

Think of the stack like bookmarks:

```
I went left from this node.
I may need to come back here later.
```

---

10. Important Variables in the Stack Algorithm

The preorder construction algorithm commonly uses these variables:

```
pre[i] = next preorder value
p = current node
t = new temporary node
stk = stack of ancestor nodes
```

`pre[i]`

This is the next value from the preorder array that we are trying to place in the tree.

Example:

```
pre = 30, 20, 10, 15, 25, 40, 50, 45
```

If `i = 3`, then:

```
pre[i] = 15
```

`p`

`p` points to the current node.

We compare `pre[i]` with `p->data`.

`t`

`t` is used when creating a new node.

Example:

```
t = new Node{nullptr, pre[i], nullptr};
```

`stk`

`stk` is a stack of node pointers.

It stores ancestors that we may need to return to later.

---

11. The Three Cases in the Stack Algorithm

At each step, we look at:

```
current node p
next value pre[i]
stack top
```

Then we choose one of three cases.

The three cases are:

1. Case A: Go left.
2. Case B: Go right.
3. Case C: Backtrack.

---

12. Case A: Go Left

Condition

If the next preorder value is smaller than the current node:

```
pre[i] < p->data
```

Then the next value becomes the left child of `p`.

Action

1. Create a new node.
2. Attach it as `p->lchild`.
3. Push `p` onto the stack.
4. Move `p` to the new node.
5. Increment `i` because the value has been placed.

In simple words:

```
The next value is smaller, so it goes left.
Before going left, save the current node in the stack.
```

Why push the current node?

When we go left, we may later need to come back and attach a right child to this current node.

So we save it.

Example

Current node:

```
p = 30
```

Next value:

```
pre[i] = 20
```

Check:

```
20 < 30
```

So `20` becomes the left child of `30`.

```
  30
 /
20
```

We push `30` onto the stack.

```
stack: 30
p = 20
```

---

13. Case B: Go Right

Condition

If the next preorder value is greater than the current node:

```
pre[i] > p->data
```

Then it may become the right child.

But there is another rule.

The value must also be smaller than the nearest ancestor on the stack.

So the full condition is:

```
pre[i] > p->data && (stk.empty() || pre[i] < stk.top()->data)
```

Meaning

The next value can go right if:

1. It is bigger than the current node.
2. It does not break the ancestor limit.

Why check the stack top?

Because when we are inside a left subtree, all values must stay below the ancestor value.

Example:

```
    30
   /
 20
 /
10
```

Here, node `10` is inside the left subtree of `20` and also inside the left subtree of `30`.

If we want to attach a right child to `10`, that right child must be:

```
greater than 10
smaller than 20
```

So `15` is allowed:

```
15 > 10
15 < 20
```

But `25` is not allowed under `10`:

```
25 > 10
25 < 20 is false
```

Action

If Case B is true:

1. Create a new node.
2. Attach it as `p->rchild`.
3. Move `p` to the new node.
4. Increment `i`.
5. Do not push `p` in the simple right case.

Example

Current node:

```
p = 10
```

Stack top:

```
20
```

Next value:

```
15
```

Check:

```
15 > 10
15 < 20
```

So `15` becomes right child of `10`.

```
  20
 /
10

  15
```

---

14. Case C: Backtrack

Condition

Case C happens when the next value is too large to fit in the current allowed range.

This usually means:

```
pre[i] is greater than p->data,
but it is not smaller than the stack top.
```

So the value cannot be placed under the current node.

Action

Backtrack by popping from the stack:

```
p = stk.top();
stk.pop();
```

Very important:

```
Do not increment i when popping.
```

Why?

Because popping does not place the value.

It only moves `p` back to an ancestor so we can try again.

Example

Current node:

```
p = 15
```

Stack top:

```
20
```

Next value:

```
25
```

Check:

```
25 > 15
25 < 20 is false
```

So `25` cannot go under `15`.

Backtrack:

```
p = 20
pop 20
```

Now check `25` again with the new current node.

Current node:

```
p = 20
```

Stack top:

```
30
```

Check:

```
25 > 20
25 < 30
```

Now it fits.

So `25` becomes right child of `20`.

---

15. The Infinity Rule

The stack top gives us an upper limit.

But what if the stack is empty?

If the stack is empty, there is no ancestor upper limit.

So we treat the upper limit like infinity.

That means:

```
If stack is empty, any value greater than the current node can go right.
```

Example

Current node:

```
p = 30
```

Stack:

```
empty
```

Next value:

```
40
```

Check:

```
40 > 30
stack is empty
```

So `40` becomes the right child of `30`.

```
30

  40
```

This is called the infinity rule because an empty stack means there is no upper boundary.

---

16. Full Dry Run: Build BST from Preorder

We will build a BST from this preorder:

```
30, 20, 10, 15, 25, 40, 50, 45
```

Step 0: Create the root

The first value is always the root.

```
30
```

So:

```
root = 30
p = 30
stack = empty
```

Tree:

```
30
```

Step 1: Place 20

Next value:

```
20
```

Current node:

```
p = 30
```

Check:

```
20 < 30
```

So this is Case A: go left.

Action:

- Attach `20` as left child of `30`.
- Push `30`.
- Move `p` to `20`.

Tree:

```
  30
 /
20
```

Stack:

```
30
```

Current node:

```
p = 20
```

Step 2: Place 10

Next value:

```
10
```

Current node:

```
p = 20
```

Check:

```
10 < 20
```

So this is Case A: go left.

Action:

- Attach `10` as left child of `20`.
- Push `20`.
- Move `p` to `10`.

Tree:

```
    30
   /
```

```
 20
 /
10
```

Stack:

```
30, 20
```

Current node:

```
p = 10
```

Step 3: Place 15

Next value:

```
15
```

Current node:

```
p = 10
```

Stack top:

```
20
```

Check:

```
15 > 10
15 < 20
```

So this is Case B: go right.

Action:

- Attach `15` as right child of `10`.
- Move `p` to `15`.
- Do not push.

Tree:

```
    30
   /
 20
 /
10

  15
```

Stack:

```
30, 20
```

Current node:

```
p = 15
```

Step 4: Try to place 25

Next value:

```
25
```

Current node:

```
p = 15
```

Stack top:

```
20
```

Check:

```
25 > 15
25 < 20 is false
```

So `25` cannot go right of `15`.

This is Case C: backtrack.

Action:

```
p = 20
pop 20
```

Do not move to the next preorder value yet.

We still need to place `25`.

Stack now:

```
30
```

Current node now:

```
p = 20
```

Step 5: Place 25

Now try `25` again.

Current node:

```
p = 20
```

Stack top:

```
30
```

Check:

```
25 > 20
25 < 30
```

So this is Case B: go right.

Action:

- Attach `25` as right child of `20`.
- Move `p` to `25`.

Tree:

```
     30
    /
  20
  /
10  25

   15
```

Stack:

```
30
```

Current node:

```
p = 25
```

Step 6: Try to place 40

Next value:

```
40
```

Current node:

```
p = 25
```

Stack top:

```
30
```

Check:

```
40 > 25
40 < 30 is false
```

So `40` cannot go right of `25`.

This is Case C: backtrack.

Action:

```
p = 30
pop 30
```

Do not increment the preorder index.

Stack now:

```
empty
```

Current node:

```
p = 30
```

Step 7: Place 40

Try `40` again.

Current node:

```
p = 30
```

Stack:

```
empty
```

Check:

```
40 > 30
stack empty, so no upper limit
```

This is Case B with the infinity rule.

Action:

- Attach `40` as right child of `30`.
- Move `p` to `40`.

Tree:

```
    30
   /
 20     40
 /
10 25

  15
```

Stack:

```
empty
```

Current node:

```
p = 40
```

Step 8: Place 50

Next value:

```
50
```

Current node:

```
p = 40
```

Stack:

```
empty
```

Check:

```
50 > 40
stack empty, so no upper limit
```

So this is Case B: go right.

Action:

- Attach `50` as right child of `40`.
- Move `p` to `50`.

Tree:

```
    30
   /
 20    40
 / \
10 25    50

  15
```

Current node:

```
p = 50
```

Step 9: Place 45

Next value:

```
45
```

Current node:

```
p = 50
```

Check:

```
45 < 50
```

So this is Case A: go left.

Action:

- Attach `45` as left child of `50`.
- Push `50`.
- Move `p` to `45`.

Final tree:

```
        30
       /
     20      40
    /   \
  10    25     50
    \          /
    15        45
```

---

17. Dry Run Table

Preorder:

```
30, 20, 10, 15, 25, 40, 50, 45
```

Next value	Current `p`	Stack top	Case	Action
30	none	empty	root	Create root `30`
20	30	empty	A: left	Attach `20` left of `30`, push `30`

Next value	Current `p`	Stack top	Case	Action
10	20	30	A: left	Attach `10` left of `20`, push `20`
15	10	20	B: right	Attach `15` right of `10`
25	15	20	C: backtrack	Pop `20`, do not increment index
25	20	30	B: right	Attach `25` right of `20`
40	25	30	C: backtrack	Pop `30`, do not increment index
40	30	empty	B: right	Attach `40` right of `30`
50	40	empty	B: right	Attach `50` right of `40`
45	50	empty	A: left	Attach `45` left of `50`, push `50`

---

18. Final Tree from the Dry Run

The final BST is:

```
        30
       /
     20     40
    /   \
  10    25     50
    \          /
    15        45
```

If we print this using in-order traversal, we get:

```
10 15 20 25 30 40 45 50
```

This output is sorted.

That proves the BST property is correct.

---

19. Complete C++ Program

```cpp
#include <iostream>
#include <stack>
```

```cpp
#include <vector>
using namespace std;

struct Node {
    Node *lchild;
    int data;
    Node *rchild;
};

Node* newNode(int value) {
    return new Node{nullptr, value, nullptr};
}

Node* createFromPreorder(const vector<int>& pre) {
    if (pre.empty()) {
        return nullptr;
    }

    int i = 0;

    Node *root = newNode(pre[i++]);
    Node *p = root;
    Node *t = nullptr;

    stack<Node*> stk;

    while (i < (int)pre.size()) {

        // Case A: next value is smaller, so it becomes the left child
        if (pre[i] < p->data) {
            t = newNode(pre[i++]);
            p->lchild = t;
            stk.push(p);
            p = t;
        }

        // Case B: next value is greater and inside the allowed range
        else if (pre[i] > p->data && (stk.empty() || pre[i] < stk.top()->data))
{
            t = newNode(pre[i++]);
            p->rchild = t;
            p = t;
        }

        // Case C: value is too large for this area, so backtrack
        else {
            p = stk.top();
            stk.pop();
```

```
        }
    }

    return root;
}

void inorder(Node *p) {
    if (p != nullptr) {
        inorder(p->lchild);
        cout << p->data << " ";
        inorder(p->rchild);
    }
}

int height(Node *p) {
    if (p == nullptr) {
        return 0;
    }

    int x = height(p->lchild);
    int y = height(p->rchild);

    return (x > y) ? x + 1 : y + 1;
}

int main() {
    vector<int> pre = {30, 20, 10, 15, 25, 40, 50, 45};

    Node *root = createFromPreorder(pre);

    cout << "In-order after preorder construction: ";
    inorder(root);

    cout << "\nHeight of preorder tree: " << height(root) << endl;

    return 0;
}
```

---

20. Program Output

The output will be:

```
In-order after preorder construction: 10 15 20 25 30 40 45 50
Height of preorder tree: 4
```

Meaning:

- The in-order traversal is sorted.
- Therefore the tree is a valid BST.
- The height is `4` if we count height by number of levels.

---

21. Code Explanation

Node structure

```
struct Node {
    Node *lchild;
    int data;
    Node *rchild;
};
```

Each node has:

- a left child pointer,
- a value,
- a right child pointer.

Create a new node

```
Node* newNode(int value) {
    return new Node{nullptr, value, nullptr};
}
```

This creates a new node with:

```
left child = nullptr
data = value
right child = nullptr
```

Empty preorder check

```
if (pre.empty()) {
    return nullptr;
}
```

If there are no values, there is no tree.

Create the root

```
Node *root = newNode(pre[i++]);
```

The first preorder value becomes the root.

Then `i++` moves to the next preorder value.

Current pointer

```
Node *p = root;
```

`p` points to the current node.

Stack

```
stack<Node*> stk;
```

The stack stores ancestors.

Case A code

```
if (pre[i] < p->data) {
    t = newNode(pre[i++]);
    p->lchild = t;
    stk.push(p);
    p = t;
}
```

This means:

- If the next value is smaller, attach it left.

- Push the current node because we may need to come back later.
- Move to the new node.

Case B code

```
else if (pre[i] > p->data && (stk.empty() || pre[i] < stk.top()->data)) {
    t = newNode(pre[i++]);
    p->rchild = t;
    p = t;
}
```

This means:

- If the next value is greater than the current node, it may go right.
- But it must also be smaller than the nearest ancestor on the stack.
- If the stack is empty, there is no upper limit.

Case C code

```
else {
    p = stk.top();
    stk.pop();
}
```

This means:

- The next value does not fit under the current node.
- Move back to an ancestor.
- Do not increment `i` because the value has not been placed yet.

---

22. Complexity of Preorder Construction

For the stack-based preorder method:

```
Time complexity: O(n)
Space complexity: O(n)
```

Where:

```
n = number of nodes
```

Why time is O(n)

Each preorder value is processed once.

Each node is created once.

Each node can be pushed onto the stack once.

Each node can be popped from the stack once.

So the total work is linear.

Therefore:

```
Time complexity = O(n)
```

Why space is O(n)

The stack may store many ancestors.

In the worst case, it can store up to `n` nodes.

Therefore:

```
Space complexity = O(n)
```

---

23. Part Two of Phase 5: Main Drawback of an Ordinary BST

Now we discuss the biggest weakness of a normal BST.

The weakness is:

```
A normal BST does not automatically control its height.
```

This matters because most BST operations depend on height.

The general rule is:

```
BST operation time = O(h)
```

Where:

```
h = height of the tree
```

Search, insertion, and deletion usually follow one path from root downward.

So if the height is small, the operation is fast.

If the height is large, the operation is slow.

---

24. Balanced BST

A balanced BST has small height.

Example:

```
        40
       /
     20     60
    / \     /
  10  30   50 70
```

This tree is nicely spread out.

If we search for `70`, the path is:

```
40 -> 60 -> 70
```

Only 3 nodes are checked.

For a balanced BST:

```
h is about log n
```

So operations are:

```
O(log n)
```

This is the good case.

---

25. Skewed BST

A skewed BST is shaped like a linked list.

Example:

```
10
  20
    30
      40
        50
          60
            70
```

If we search for `70`, the path is:

```
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> 70
```

We must check every node.

For a skewed BST:

```
h can become n
```

So operations become:

```
O(n)
```

This is the bad case.

---

26. Same Values, Different Insertion Order

The same values can create different BST shapes.

Use these values:

```
10, 20, 30, 40, 50, 60, 70
```

Balanced-like insertion order

If we insert in this order:

```
40, 20, 60, 10, 30, 50, 70
```

The tree becomes:

```
        40
       /
     20     60
    / \     /
  10  30  50 70
```

This tree has small height.

Operations are fast.

Sorted insertion order

If we insert in this order:

```
10, 20, 30, 40, 50, 60, 70
```

The tree becomes:

```
10
  20
    30
      40
        50
          60
            70
```

This tree is skewed.

It behaves like a linked list.

Operations become slow.

---

27. Height Definition Note

Different books sometimes define height differently.

Height by levels

In earlier phases, we counted height by number of levels.

Using this definition:

```
A single node has height 1.
An empty tree has height 0.
```

For the skewed tree with 7 nodes:

```
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> 70
```

The height is:

```
7
```

Height by edges

Some sources count height by number of edges.

Using this definition:

```
A single node has height 0.
```

For the same 7-node skewed tree, the height is:

```
6
```

Both definitions are common.

The conclusion is the same:

```
Sorted input creates a very tall BST.
```

---

28. Why the Main Drawback Happens

A normal BST uses the insertion order given by the user.

It does not rearrange itself.

If the insertion order is good, the tree may be balanced.

If the insertion order is bad, the tree may become skewed.

The BST itself does not fix the problem.

That is the main drawback:

```
A normal BST has no automatic height control.
```

---

29. Effect on Search, Insert, and Delete

Most BST operations depend on height.

Search

Search follows one path.

Balanced tree:

```
O(log n)
```

Skewed tree:

```
O(n)
```

Insert

Insertion first searches for the correct `NULL` position.

Balanced tree:

```
O(log n)
```

Skewed tree:

```
O(n)
```

Delete

Deletion first searches for the node.

Then it may find predecessor or successor.

Balanced tree:

```
O(log n)
```

Skewed tree:

```
O(n)
```

So the simple rule is:

```
BST operations are O(h).
```

Then:

```
If balanced, h = log n.
If skewed, h = n.
```

---

30. Why AVL Trees Are Mentioned

AVL trees are mentioned because they solve the height problem.

An AVL tree is a self-balancing BST.

It still follows the BST rule:

```
left values < node value < right values
```

But it also checks balance.

If the tree becomes too unbalanced, the AVL tree rotates nodes to fix the shape.

The purpose is to keep height small.

So:

```
Ordinary BST:
Fast if balanced naturally.
Slow if skewed.
```

```
AVL tree:
Actively keeps itself balanced.
```

In Phase 5, you only need the motivation.

You do not need full AVL insertion and deletion code yet.

---

31. Complete Program Including Preorder Construction and Drawback Example

```
#include <iostream>
#include <stack>
#include <vector>
using namespace std;

struct Node {
    Node *lchild;
    int data;
    Node *rchild;
};

Node* newNode(int value) {
    return new Node{nullptr, value, nullptr};
}

Node* createFromPreorder(const vector<int>& pre) {
    if (pre.empty()) {
        return nullptr;
    }

    int i = 0;
    Node *root = newNode(pre[i++]);
    Node *p = root;
    Node *t = nullptr;
    stack<Node*> stk;

    while (i < (int)pre.size()) {
        if (pre[i] < p->data) {
            t = newNode(pre[i++]);
            p->lchild = t;
            stk.push(p);
            p = t;
        }
        else if (pre[i] > p->data && (stk.empty() || pre[i] < stk.top()->data))
{
            t = newNode(pre[i++]);
            p->rchild = t;
            p = t;
        }
```

```
        else {
            p = stk.top();
            stk.pop();
        }
    }

    return root;
}

void inorder(Node *p) {
    if (p != nullptr) {
        inorder(p->lchild);
        cout << p->data << " ";
        inorder(p->rchild);
    }
}

int height(Node *p) {
    if (p == nullptr) {
        return 0;
    }

    int x = height(p->lchild);
    int y = height(p->rchild);

    return (x > y) ? x + 1 : y + 1;
}

Node* insert(Node *root, int key) {
    if (root == nullptr) {
        return newNode(key);
    }

    if (key < root->data) {
        root->lchild = insert(root->lchild, key);
    }
    else if (key > root->data) {
        root->rchild = insert(root->rchild, key);
    }

    return root;
}

Node* buildByInsertion(const vector<int>& keys) {
    Node *root = nullptr;

    for (int key : keys) {
        root = insert(root, key);
```

```
    }

    return root;
}

int main() {
    vector<int> pre = {30, 20, 10, 15, 25, 40, 50, 45};
    Node *fromPre = createFromPreorder(pre);

    cout << "In-order after preorder construction: ";
    inorder(fromPre);
    cout << "\nHeight of preorder tree: " << height(fromPre) << "\n\n";

    vector<int> mixed = {40, 20, 60, 10, 30, 50, 70};
    vector<int> sorted = {10, 20, 30, 40, 50, 60, 70};

    Node *balancedLike = buildByInsertion(mixed);
    Node *skewed = buildByInsertion(sorted);

    cout << "Height from balanced-like insertion order: " <<
height(balancedLike) << "\n";
    cout << "Height from sorted insertion order: " << height(skewed) << "\n";

    return 0;
}
```

---

32. Expected Output

```
In-order after preorder construction: 10 15 20 25 30 40 45 50
Height of preorder tree: 4

Height from balanced-like insertion order: 3
Height from sorted insertion order: 7
```

Meaning:

- The preorder construction produced a valid BST.
- The in-order output is sorted.
- The balanced-like insertion order has smaller height.
- The sorted insertion order creates a tall skewed tree.

---

33. Common Beginner Mistakes in Phase 5

Mistake 1: Thinking preorder alone can build any binary tree

Wrong idea:

```
If I have preorder, I can build any binary tree.
```

Correct idea:

```
Preorder alone is not enough for a normal binary tree.
It works here because we are building a BST.
```

The BST rule gives extra information.

Mistake 2: Forgetting that the first preorder value is the root

Preorder starts with the current node.

So the first value is always the root.

Example:

```
Preorder: 30, 20, 10, 15
```

Root:

```
30
```

Mistake 3: Not using the stack when going left

When going left, push the current node.

Wrong idea:

```
Just move left without saving anything.
```

Problem:

You may need to come back later to attach a right child.

Correct idea:

```
Before going left, push the current node onto the stack.
```

Mistake 4: Pushing when going right unnecessarily

In the simple right case, we do not push the current node.

When going left, push because you may need to return.

When going right within the valid range, just attach and move.

Mistake 5: Forgetting the ancestor upper bound

Wrong condition:

```
pre[i] > p->data
```

This is not enough.

Correct condition:

```
pre[i] > p->data && (stk.empty() || pre[i] < stk.top()->data)
```

Why?

Because a node inside a left subtree must still be smaller than its ancestor.

Mistake 6: Incrementing `i` after popping

Wrong:

```
p = stk.top();
stk.pop();
i++;
```

Correct:

```
p = stk.top();
stk.pop();
```

Do not increment `i`.

Why?

Because the value has not been placed yet.

You only moved back to an ancestor.

Mistake 7: Forgetting the infinity rule

When the stack is empty, there is no ancestor upper limit.

So this condition is allowed:

```
stk.empty() || pre[i] < stk.top()->data
```

If `stk.empty()` is true, the value can go right if it is greater than the current node.

Mistake 8: Saying BST operations are always O(log n)

Wrong:

```
BST search, insertion, and deletion are always O(log n).
```

Correct:

```
BST operations are O(h).
```

Then explain:

```
Balanced tree: O(log n)
Skewed tree: O(n)
```

Mistake 9: Thinking sorted input is good for a BST

For a normal BST, sorted input is bad.

Example:

```
10, 20, 30, 40, 50
```

This creates:

```
10

  20

    30

      40

        50
```

This is skewed.

It behaves like a linked list.

---

34. Key Rules to Remember

Preorder rule

```
Preorder = Node, Left, Right
```

BST rule

```
left values < node value < right values
```

First preorder value

```
The first preorder value is the root.
```

Case A: Go left

```
If pre[i] < p->data:
    attach as left child
    push p
    move p to new node
    increment i
```

Case B: Go right

```
If pre[i] > p->data and value is inside ancestor limit:
    attach as right child
    move p to new node
    increment i
```

Full condition:

```
pre[i] > p->data && (stk.empty() || pre[i] < stk.top()->data)
```

Case C: Backtrack

```
If value does not fit:
    p = stack top
    pop stack
    do not increment i
```

Infinity rule

```
If the stack is empty, there is no upper limit.
```

Complexity of preorder construction

```
Time: O(n)
Space: O(n)
```

Main drawback of ordinary BST

```
A normal BST does not automatically control height.
```

Operation complexity

```
BST operations are O(h).
Balanced tree: O(log n)
Skewed tree: O(n)
```

---

35. Mini Practice 1: Identify the Root

Given preorder:

```
50, 30, 20, 40, 70, 60, 80
```

Question:

What is the root?

Answer:

```
50
```

Reason:

The first preorder value is always the root.

---

36. Mini Practice 2: First Few Insertions from Preorder

Given preorder:

```
50, 30, 20
```

Step 1:

```
50 is root
```

Step 2:

```
30 < 50
```

So `30` goes left.

Step 3:

```
20 < 30
```

So `20` goes left of `30`.

Tree:

```
    50
   /
 30
 /
20
```

---

37. Mini Practice 3: Backtracking

Suppose part of the tree is:

```
    50
   /
 30
 /
20
```

Stack:

```
50, 30
```

Current node:

```
p = 20
```

Next value:

```
40
```

Check:

```
40 > 20
40 < 30 is false
```

So `40` cannot go right of `20`.

Backtrack:

```
p = 30
pop 30
```

Now stack top is `50`.

Check:

```
40 > 30
40 < 50
```

So `40` becomes right child of `30`.

Tree:

```
    50
   /
 30
 /
20  40
```

---

38. Mini Practice 4: Balanced vs Skewed

Use keys:

```
10, 20, 30, 40, 50
```

If inserted in sorted order:

```
10, 20, 30, 40, 50
```

Tree:

```
10

  20

    30

      40

        50
```

This is skewed.

Search for `50` takes:

```
10 -> 20 -> 30 -> 40 -> 50
```

That is slow.

A better insertion order is:

```
30, 20, 40, 10, 50
```

Possible tree:

```
    30
   /
 20    40
 /
10      50
```

This is more balanced.

---

39. Phase 5 Final Summary

Phase 5 teaches how to generate a BST from preorder traversal and why ordinary BSTs can become slow.

Preorder construction summary

Preorder traversal is:

```
Node, Left, Right
```

The first preorder value becomes the root.

The remaining values are placed using the BST rule.

The stack stores ancestors so we can backtrack when needed.

There are three main cases:

```
Case A: smaller value -> go left and push current node
Case B: larger valid value -> go right
Case C: value too large for current range -> pop and backtrack
```

The stack top acts like an upper boundary.

If the stack is empty, there is no upper boundary. This is the infinity rule.

The algorithm runs in:

```
O(n) time
O(n) space
```

Main drawback summary

A normal BST is fast only when its height is small.

General rule:

```
BST operations = O(h)
```

If the tree is balanced:

```
O(log n)
```

If the tree is skewed:

```
O(n)
```

The main problem is:

```
A normal BST does not automatically balance itself.
```

Sorted insertion order can create a linked-list-like tree.

AVL trees solve this problem by keeping the tree balanced.

---

40. Final Mental Model

Think of preorder construction like this:

```
Preorder tells the order of nodes.
BST rule tells where each value belongs.
The stack remembers ancestors.
```

At each value, ask:

```
Is it smaller than current node?
    Go left, push current node.

Is it greater and inside allowed range?
    Go right.

Is it too large for this area?
    Pop from stack and try again.
```

Think of the BST drawback like this:

```
A BST is fast only if it stays short.
A normal BST does not guarantee that.
Bad insertion order can make it tall.
A tall BST behaves like a linked list.
```

That is the heart of Phase 5.

---

41. What You Should Be Able to Do After Phase 5

After studying Phase 5, you should be able to:

- Explain what preorder traversal means.
- State the preorder order: Node, Left, Right.
- Explain why the first preorder value becomes the root.
- Explain why preorder alone cannot build every binary tree.
- Explain why preorder can build a BST.
- Use the BST rule while reading preorder values.
- Explain why a stack is needed.
- Explain what `p`, `t`, `pre[i]`, and `stk` mean.
- Explain Case A: smaller value goes left.
- Explain Case B: larger valid value goes right.
- Explain Case C: out-of-range value causes backtracking.
- Explain the infinity rule.
- Dry run preorder construction for a small example.
- Write or understand the C++ preorder construction code.
- Explain why preorder construction is `O(n)` time.
- Explain why stack space can be `O(n)`.
- Explain why ordinary BSTs can become skewed.
- Compare balanced and skewed BSTs.
- Explain why BST operations are `O(h)`.
- Explain when BST operations become `O(log n)`.
- Explain when BST operations become `O(n)`.

- Explain why AVL trees are introduced after ordinary BSTs.

---

42. One-Page Revision Sheet

Preorder

```
Node, Left, Right
```

First value

```
First preorder value = root
```

BST rule

```
left < node < right
```

Stack purpose

```
The stack remembers ancestors for backtracking.
```

Case A

```
pre[i] < p->data
Go left.
Push p.
Move p to new node.
Increment i.
```

Case B

```
pre[i] > p->data and value is inside ancestor boundary
Go right.
Move p to new node.
Increment i.
```

Case C

```
Value does not fit current range.
Pop stack.
Do not increment i.
Try same value again.
```

Infinity rule

```
If stack is empty, there is no upper boundary.
```

Preorder construction complexity

```
Time: O(n)
Space: O(n)
```

Main drawback of normal BST

```
No automatic height control.
```

BST operation time

```
O(h)
```

Balanced BST

```
h ≈ log n
operations = O(log n)
```

Skewed BST

```
h ≈ n
operations = O(n)
```

AVL motivation

```
AVL trees keep BSTs balanced.
```